

Code review: An initial assignment

[New Attempt](#)

- Due Feb 7 by 11pm
- Points 10
- Submitting a text entry box, a website url, or a file upload
- Available after Jan 31 at 1pm

Initial approaches to code review

The primary purpose of this assignment is to demonstrate work in code review. In doing so, you will show that you can take specified code and do the following:

- Confirm that it functions according to specifications
- Raise questions about programming choices
- Suggest changes for efficiency and clarity
- Recognize potential errors resulting from edge cases
- Compare several different functional solutions and identify under what circumstances one might be preferred
- Decide whether proposed changes meet specifications and/or organizational standards
- Propose fixes to problems that prevent a code from running correctly on every run
- Identify and compliment well-structured and commented code

This assignment **must** be submitted individually. You may consult with others in the class, but you must submit your own repository with your own review and modifications to the code.

Introduction

Reviewing working code, identifying what works well, and making it better is an important task for any software developer. Doing so typically improves our own programming skills as well.

Every organization has its own code review practices. We will be using a reasonable set of guidelines for code review, but that doesn't mean you will find exactly the same procedures in every code review you do. And as you review more code, you will establish your own sense of what good code looks like.

The guidelines I use here are based on my experience as a programmer, guided by several sources, such as the book *Code Complete 2* by Steve McConnell, the online MIT course on software construction, and very useful checklists from the software engineers Karl Wieggers (whose background is as a chemist!) and Gareth Wilson. Some of these lists and guidelines date from 20–25 years ago, and I have updated pieces of them to conform to more current programming language. But the fundamentals are still the same.

The next section contains those fundamentals. In that, you should see a number of things that I have noted in our class discussion or in comments on your assignments (such as avoiding “magic numbers” in your code). **Read through that section completely before continuing to the next step.**

The sections after that are the exercises you will complete for this project. You will review one of my (very recently generated) code sets based on provided guidelines.

Basic principles of code review in CTIS 310

Why do we do code review? We can start by summarizing the objectives at the beginning of this assignment. We want code that

- Is as bug-free as possible;
- Clarifies its purpose so you and others can understand it in the future
- Can be maintained without simply starting over

Based on the sources I listed above, here are the items I'll ask you to consider when doing a code review.

Documentation

- Are there clear and correct Javadoc comments for the class, method, and/or attributes? Note that both public and private methods and attributes should have Javadoc comments.
 - For this assignment, I'll tell you that there are definitely not the Javadoc comments present that there should be.

Code structure

- Does the code satisfy all provided specifications?
- Is the code consistently formatted? (Note: For this course, we use the results of the Format Document... command in Visual Studio Code as our correct formatting style for Java programs.)
- Do all methods have a clear purpose?
- Is there any debugging code still present that should be commented out or otherwise modified so that it does not run in production mode?
- What repeated code is present that could become a single method? (This principle is known as **DRY**: Don't repeat yourself.)
- Are there any “magic number” constants that should be redefined as `final` variables (as in Java; other languages will have a different keyword)?
 - Should have any of them be `static` variables that can be accessed directly by other objects?
- Is there any complexity that can be simplified by the use of multiple, clearer methods?

Variables

- Do all variables (both attributes and other local variables) have reasonable types and identifiers?
- Does each variable have a single purpose in its scope?
 - It might help to have an example here: In a method, you might have a variable `index` that at one point in the method keeps track of the index of one array, but then is used later in the same

method for a different array. The code can work, but it can be difficult for someone else to understand and modify correctly.

- Whenever the code assigns a value to a variable, does the code ensure *type consistency*? For example, does the code use casting appropriately?
- Are there variables that can be removed from the code because they are redundant or unused?

Arithmetic operations

- Are there any places that floating-point numbers are compared for equality? If so, how can those comparisons be rewritten to be more robust?
- Are divisions tested for divide-by-zero?

Loops and conditional statements

- Are there any errors in the nesting of loops and conditional statements?
- In an `if-else` chain or `switch` statement, are the most common conditions tested first?
- Does each `if-else` chain or `switch` statement address all possible cases? (Note the importance of the final `else` and `default` blocks.)
- Can readability and robustness be improved by converting an `if-else` statement to a `switch`?
- Do loops (for, while, or for-each) have proper index initialization statements?
- Are loop termination conditions guaranteed to be satisfied when the program runs?
- Are there statements that can be moved outside of the loop (because they only need to run once) or conditional statement (because they run no matter what the result of the conditional statement is)?

General programming practice

- Are indexes tested against array bounds?
- Are all return values assigned and returned correctly?
- Do methods avoid the use of print statements and instead return values or strings?
- Does each statement operate on the correct data?
- Does the code provide reasonable results for a wide range of test cases, especially including any potential edge cases?

Other code review items

- Are there other issues with the code that you feel are worth pointing out even though they don't obviously fit into one of the above items?

Code review exercise

This exercise is complex because you will be reading specifications for a project and then evaluating code implementing the project that you did not write.

Complete the following steps.

- **Fork** my [Card Driver Project repository](https://github.com/rwhitnell/cardgamedriver)  (<https://github.com/rwhitnell/cardgamedriver>). By forking the project, you are creating a copy of the repository in your GitHub account. You will submit the link to this repository for this assignment after you complete the remaining steps.
- Use Visual Studio Code to **clone** and open your forked project repository.

- You will see a project with several classes and a README.md file. The README file contains the specifications that the project is intended to satisfy.
- Read through the specifications for the project.
- Run the program several times to see what the output is. Examine the output and see how it connects to what's given in the specifications.
- Add correct and complete Javadoc comments for any constructors or methods more complex than getters, setters or other methods that have a method body with only one line.
- Add a codereview.md file to your project that uses well-formatted Markdown to address specific code review items for each part of the assignment.
 - This file **must** demonstrate use of Markdown to provide a well-formatted document.

Reviewing implementation of a specification and class definitions

The repository you forked contains my early attempts at building two card games, Blackjack and Lamarckian Poker, with the ultimate purpose of testing random and semi-random approaches to playing these games. I wrote this implementation very quickly, and I have not reviewed it in detail. So now you will.

- Before starting, review the specifications in the README.md. Recall that specifications are promises that the program must adhere to. That's sometimes difficult because the specifications aren't always clear about all the promises they should make. And sometimes specifications just lie.
- Run the program and compare the output to what's described about the driver program in the specifications. Note that you have the ability to make changes, try new tests, and otherwise modify the program. But keep track of what changes you make and what was the original code. Commits can help you here.
 - You will run into a bug that causes the program to crash. That's addressed in the next part of this assignment.
- Now start looking through each class in the code to see what it does and how that implements the specifications. I have not given you much help with comments (primarily because I ran out of time), so you may want to add comments to help you. Copilot might help here, but be sure you review and correct anything it does for you.
- Identify all code review items from the list above that may be relevant to this project. Add the list of relevant items to your codereview.md file.
- For each code review item you've identified, add text to your codereview.md file that does the following.
 - Evaluates whether the project appears to satisfy the item under review. For example, if there are loops in any of the methods, is the index initialization done correctly?
 - Include sample output to whatever extent it helps support your conclusions.
 - For any unsatisfactory items, recommends changes that would bring the code into compliance and why you expect those changes to work.
 - You can describe these changes completely in the codereview.md file, or you can modify the existing code. If you modify code, do the following:

- Comment out the original code instead of deleting it.
- Add comments to the code that describe the changes you've made.
- Add a note to the codereview.md pointing to the code you've changed.

Tracking down a bug and proposing a fix

Code reviews are often used for getting feedback on why a program isn't working in the way it's supposed to, often because there are clear run-time errors.

When you run the program, you will occasionally get an error message that an `IllegalArgumentException` occurred in the `LamarckianPoker` class. I think I know what the bug in the program is that causes that error, but now I want you to try to find it and propose solutions. Errors that only occur occasionally are often the most difficult ones to fix, but there are ways to track it down. More specifically, you should do the following.

- Run the code so that it generates the error several times. In each case, look at the information printed out with the exception, particularly which lines of code it's pointing to. Is it always the same line or same region of the program? If so, that will give you some clues about where to start looking.
- What is the specific error that occurs, based on what the exception tells you? How could the lines that the exception points to cause that error? For example, if the error was dividing by zero (which it's not in this case), how could the variables in the lines of code you're looking at end up resulting in divide by zero?
- What conditions could arise that cause that error? How could the illegal argument that's causing the error be generated occasionally as the program runs, causing it to crash?
- How would you propose fixing the program? That might be because there is a clear logical error in the program code that led to this problem. It may also be because the specifications need to be updated to account for this issue, and then the code needs to be changed.

Once you've worked through the process of identifying the problem and considering solutions, do the following.

- In your codereview.md file, describe why the error occurs. In doing so, be sure to describe why it only occurs occasionally and, most of the time, the program runs without error.
- Describe your proposed fix to address the error. If necessary, describe what you would change in the project specifications.
- Implement your change in the code. Be sure to add a comment at any point where you change code from the original version.

Challenge I'm about 99.9% certain that this program creates garbage in the Java sense: objects that no longer have a variable referencing them at some point as the code runs. (This situation is also known as a *memory leak*.) While Java garbage collection will address these objects, it's still better if it doesn't happen in the first place. I think I know where one memory leak occurs, but there might be more. Identify one or more memory leaks, provide an analysis of the code that shows how the leak occurs, and propose and/or implement modifications to fix the leak.

Challenge As I was doing my final testing of the code in preparing this assignment, the program threw an exception in the `Blackjack` class that had never occurred before. See if you can recreate that exception and then address using the same set of steps as above. This particular exception was an `IndexOutOfBoundsException`. And, of course, you might find others, and you can choose to address those as well.

Requirements

Submit a link (or links) to your forked GitHub repository containing your revised code and your `codereview.md` file.

The assignment is considered complete if you submit all the components, the project can be downloaded and run in VS Code without error, your `codereview.md` file is present and uses Markdown properly, and the overall assignment meets at least 80% of the specifications given above.